

University of Engineering and Technology

Compiler Construction Lab Mini Compiler Project Report

Submitted By:

Member 1: Munees Tariq (2023-CS-32)

Member 2: Muhammad Mahad Sheraz (2023-CS-36)

Section: A

Supervisor:

Mam Aliya Farooq

Spring 2026

Contents

1	Introduction	2
2	Grammar Specification	2
2.1	Original and LL(1) Grammar	2
3	Module Descriptions	3
3.1	Lexical Analyzer (Lab 3)	3
3.2	Recursive Descent Parser (Lab 4)	3
3.3	Non-Recursive Predictive Parser (Lab 5)	3
3.4	LR Parser (New Module)	4
3.5	Symbol Table Manager (Lab 7, Extended)	4
3.6	Error Handler (Bonus Task)	4
4	Construction of the LR Parsing Table	4
5	Module Integration	5
6	Sample Input and Output	5
6.1	Sample Input	5
6.2	Invalid Syntax Input	5
6.3	Lexer Output	5
6.4	LR Parser Trace	6
6.5	Symbol Table Output	7
7	Limitations	7
8	References	7

1 Introduction

This report details the design and implementation of the mini compiler for the CS-471L Compiler Construction Lab final project. The compiler is built for a subset of the MicroJava programming language. It integrates all the lab modules developed during the semester, along with additional components such as an LR Parser, Symbol Table Manager, and Error Handling.

2 Grammar Specification

The language subset used is MicroJava. The grammar supports class declarations, variable declarations, methods, statements (if, while, return, print), expressions, and assignments. The original grammar, as well as the left-factored and non-left-recursive version used for top-down parsing, are specified in the accompanying documentation.

2.1 Original and LL(1) Grammar

The full grammar in BNF form can be found in docs/grammar.md.

```
# Section 1 Grammar

This project uses a compact MicroJava subset with classes, fields, methods, local declarations,
assignments, calls, loops, conditionals, returns, printing, arrays, arithmetic, relational
expressions, and character constants.

## Original BNF

The original grammar keeps expression precedence left-recursive for the LR parser.

- 'Program ::= class id { MemberList }'
- 'MemberList ::= Member MemberList | epsilon'
- 'Member ::= Type id MemberAfter'
- 'MemberAfter ::= ; | [ number ] ; | ( ParamListOpt ) Block'
- 'Type ::= int | char | boolean | void'
- 'ParamListOpt ::= ParamList | epsilon'
- 'ParamList ::= Type id ParamListTail'
- 'ParamListTail ::= , Type id ParamListTail | epsilon'
- 'Block ::= { StmtList }'
- 'StmtList ::= Stmt StmtList | epsilon'
- 'Stmt ::= Type id VarDeclAfter ; | id StmtIdTail ; | return ExprOpt ; | if ( Expr ) Stmt | while ( Expr
    ) Stmt | Block ; | print ( Expr ) ;'
- 'VarDeclAfter ::= [ number ] | epsilon'
- 'StmtIdTail ::= AssignTarget = Expr | ( ArgListOpt )'
- 'AssignTarget ::= [ Expr ] | epsilon'
- 'ExprOpt ::= Expr | epsilon'
- 'ArgListOpt ::= ArgList | epsilon'
- 'ArgList ::= Expr ArgListTail'
- 'ArgListTail ::= , Expr ArgListTail | epsilon'
- 'RelOp ::= == | != | < | <= | > | >='
- 'AddOp ::= + | -'
- 'MulOp ::= * | / | %'
- 'Factor ::= id FactorAfter | number | char_const | true | false | ( Expr )'
- 'FactorAfter ::= ( ArgListOpt ) | [ Expr ] | epsilon'
- 'Expr ::= Expr RelOp AddExpr | AddExpr'
- 'AddExpr ::= AddExpr AddOp Term | Term'
- 'Term ::= Term MulOp Factor | Factor'

## LL(1) Transformed Grammar

- 'Program ::= class id { MemberList }'
- 'MemberList ::= Member MemberList | epsilon'
- 'Member ::= Type id MemberAfter'
- 'MemberAfter ::= ; | [ number ] ; | ( ParamListOpt ) Block'
- 'Type ::= int | char | boolean | void'
```

```

- 'ParamListOpt ::= ParamList | epsilon'
- 'ParamList ::= Type id ParamListTail'
- 'ParamListTail ::= , Type id ParamListTail | epsilon'
- 'Block ::= { StmtList }'
- 'StmtList ::= Stmt StmtList | epsilon'
- 'Stmt ::= Type id VarDeclAfter ; | id StmtIdTail ; | return ExprOpt ; | if ( Expr ) Stmt | while ( Expr
    ) Stmt | Block | ; | print ( Expr ) ;'
- 'VarDeclAfter ::= [ number ] | epsilon'
- 'StmtIdTail ::= AssignTarget = Expr | ( ArgListOpt )'
- 'AssignTarget ::= [ Expr ] | epsilon'
- 'ExprOpt ::= Expr | epsilon'
- 'ArgListOpt ::= ArgList | epsilon'
- 'ArgList ::= Expr ArgListTail'
- 'ArgListTail ::= , Expr ArgListTail | epsilon'
- 'Expr ::= RelExpr'
- 'RelExpr ::= AddExpr RelExprTail'
- 'RelExprTail ::= RelOp AddExpr | epsilon'
- 'RelOp ::= == | != | < | <= | > | >='
- 'AddExpr ::= Term AddExprTail'
- 'AddExprTail ::= AddOp Term AddExprTail | epsilon'
- 'AddOp ::= + | -'
- 'Term ::= Factor TermTail'
- 'TermTail ::= MulOp Factor TermTail | epsilon'
- 'MulOp ::= * | / | %'
- 'Factor ::= id FactorAfter | number | char_const | true | false | ( Expr )'
- 'FactorAfter ::= ( ArgListOpt ) | [ Expr ] | epsilon'

Transformation notes:
- Direct left recursion in expressions was replaced by 'AddExprTail', 'TermTail', and related tail non-
  terminals.
- Common prefixes such as member declarations and statement forms were left-factored.
- Empty productions are written as 'epsilon'.

```

3 Module Descriptions

3.1 Lexical Analyzer (Lab 3)

The lexical analyzer (`src/lexer.c`) is responsible for tokenizing the input source program. It reads characters using a double buffering technique and groups them into logical tokens such as keywords, identifiers, numbers, operators, and punctuation. It ignores whitespace and comments and keeps track of the line and column number for each token to assist in error reporting.

3.2 Recursive Descent Parser (Lab 4)

The recursive descent parser (`src/recursive_descent_parser.c`) is a top-down parser implementing a set of mutually recursive functions, one for each non-terminal in the grammar. It uses the left-factored and non-left-recursive version of the grammar to avoid infinite loops and ambiguity.

3.3 Non-Recursive Predictive Parser (Lab 5)

The predictive parser (`src/predictive_parser.c`) is an LL(1) top-down parser that avoids recursion by using an explicit stack and a parsing table. The FIRST and FOLLOW sets used to construct the LL(1) parsing table can be found in `docs/first_follow.md`, and the table itself is in `docs/ll1_table.md`.

3.4 LR Parser (New Module)

The LR parser (`src/lr_parser.c`) is a bottom-up SLR(1) parser. It uses a stack of states and grammar symbols alongside an action and goto table. It parses the original grammar (including left-recursive rules) by performing shift and reduce operations.

3.5 Symbol Table Manager (Lab 7, Extended)

The symbol table (`src/symbol_table.c`) uses a hash-based structure for efficient lookups. It handles nested scopes using a linked scope chain. For each identifier, it stores attributes such as name, kind (e.g., variable, class, function), type, scope level, and source line number. It is integrated with the parser to populate declarations and resolve usages.

3.6 Error Handler (Bonus Task)

The error handler (`src/error_handler.c`) centrally manages lexical, syntactic, and semantic errors. It implements panic-mode recovery, preventing the parser from crashing on the first error and allowing it to report multiple issues in a single pass. Errors include detailed line and column numbers.

4 Construction of the LR Parsing Table

The SLR(1) parsing technique was chosen for the LR parser. The steps to construct the SLR(1) parsing table are as follows:

1. **Augment the Grammar:** A new start symbol is added to the grammar to explicitly define when the parser should accept the input.
2. **LR(0) Item Sets:** We construct the canonical collection of LR(0) items, determining the states of the parser.
3. **Goto Graph:** Transitions between the item sets on terminals and non-terminals are defined.
4. **Action Table:**
 - Shift actions are added for terminal transitions.
 - Reduce actions are added for completed items, but only for lookahead symbols present in the FOLLOW set of the production's left-hand side non-terminal.
 - An accept action is added for the augmented start production.
5. **Goto Table:** Non-terminal transitions are added to the Goto table.

The full LR Action and Goto tables are available in `docs/lr_table.md`.

5 Module Integration

All modules are integrated into a single unified build system. The file `src/compiler.c` acts as the main entry point (dispatcher). It processes command-line arguments to determine which module to run (lexer, recursive, predictive, lr, or all). The lexer provides a token stream that any of the three parsers can consume. The parsers communicate with the Symbol Table Manager for semantic checks and the Error Handler to report issues gracefully. The compiler can be built using the provided `Makefile` or `build.bat` script.

6 Sample Input and Output

6.1 Sample Input

A valid MicroJava program, `hello.mj`:

```
class Hello {
  void main() {
    print(1 + 2 * 3);
    return;
  }
}
```

6.2 Invalid Syntax Input

```
class Broken {
  void main() {
    int x
    x = 1;
  }
}
```

6.3 Lexer Output

1:1	KEYWORD	'class'	'class'
1:7	IDENTIFIER	'Hello'	'Hello'
1:13	PUNCTUATION	'{'	'{'
2:3	KEYWORD	'void'	'void'
2:8	IDENTIFIER	'main'	'main'
2:12	PUNCTUATION	'('	'('
2:13	PUNCTUATION	)'	)'
2:15	PUNCTUATION	'{'	'{'
3:5	KEYWORD	'print'	'print'
3:10	PUNCTUATION	'('	'('
3:11	NUMBER	'1'	'1'
3:13	OPERATOR	'+'	'+'
3:15	NUMBER	'2'	'2'
3:17	OPERATOR	'*'	'*'
3:19	NUMBER	'3'	'3'
3:20	PUNCTUATION	)'	)'
3:21	PUNCTUATION	','	','
4:5	KEYWORD	'return'	'return'

```

4:11    PUNCTUATION    ';'      ';'
5:3     PUNCTUATION    '}'      '}'
6:1     PUNCTUATION    '}'      '}'
7:1     EOF            ''       NULL

```

6.4 LR Parser Trace

For at least one sample input, we show a trace with the stack contents, the remaining input, and the action taken at each step. See `docs/hello_lr_trace_utf8.txt` for a detailed trace.

```

SLR(1) parser trace
Grammar: original left-recursive expression grammar
Step  States  Symbols  Remaining input  Action
1      [0]    []      [class id { void id ( ) { ...]  shift 2
2      [0 2]  [class]  [id { void id ( ) { print ...]  shift 3
3      [0 2 3] [class id] [{ void id ( ) { print ( ...]  shift 4
4      [0 2 3 4] [class id {] [void id ( ) { print ( number ...]  shift 11
5      [0 2 3 4 11] [class id { void] [id ( ) { print ( number + ...]  reduce r11: Type -> void
6      [0 2 3 4 7] [class id { Type] [id ( ) { print ( number + ...]  shift 14
7      [0 2 3 4 7 14] [class id { Type id] [( ) { print ( number + number ...]  shift 15
8      [0 2 3 4 7 14 15] [class id { Type id ( ] [] { print ( number + number * ...]  reduce r13:
ParamListOpt -> epsilon
9      [0 2 3 4 7 14 15 20] [class id { Type id ( ParamListOpt] [] { print ( number + number * ...]
shift 23
10     [0 2 3 4 7 14 15 20 23] [class id { Type id ( ParamListOpt )] [{ print ( number + number * number
... ]  shift 27
11     [0 2 3 4 7 14 15 20 23 27] [class id { Type id ( ParamListOpt ) {] [print ( number + number *
number ) ...]  shift 38
12     [0 2 3 4 7 14 15 20 23 27 38] [class id { Type id ( ParamListOpt ) { print] [( number + number *
number ) ; ...]  shift 50
13     [0 2 3 4 7 14 15 20 23 27 38 50] [class id { Type id ( ParamListOpt ) { print ( ] [number + number
* number ) ; return ...]  shift 60
14     [0 2 3 4 7 14 15 20 23 27 38 50 60] [class id { Type id ( ParamListOpt ) { print ( number] [+
number * number ) ; return ; ...]  reduce r53: Factor -> number
15     [0 2 3 4 7 14 15 20 23 27 38 50 55] [class id { Type id ( ParamListOpt ) { print ( Factor] [+
number * number ) ; return ; ...]  reduce r66: Term -> Factor
16     [0 2 3 4 7 14 15 20 23 27 38 50 56] [class id { Type id ( ParamListOpt ) { print ( Term] [+ number
* number ) ; return ; ...]  reduce r64: AddExpr -> Term
17     [0 2 3 4 7 14 15 20 23 27 38 50 52] [class id { Type id ( ParamListOpt ) { print ( AddExpr] [+
number * number ) ; return ; ...]  shift 75
18     [0 2 3 4 7 14 15 20 23 27 38 50 52 75] [class id { Type id ( ParamListOpt ) { print ( AddExpr +] [
number * number ) ; return ; } ...]  reduce r47: AddOp -> +
19     [0 2 3 4 7 14 15 20 23 27 38 50 52 77] [class id { Type id ( ParamListOpt ) { print ( AddExpr AddOp]
[ number * number ) ; return ; } ...]  shift 60
20     [0 2 3 4 7 14 15 20 23 27 38 50 52 77 60] [class id { Type id ( ParamListOpt ) { print ( AddExpr
AddOp number] [* number ) ; return ; } } ...]  reduce r53: Factor -> number
21     [0 2 3 4 7 14 15 20 23 27 38 50 52 77 55] [class id { Type id ( ParamListOpt ) { print ( AddExpr
AddOp Factor] [* number ) ; return ; } } ...]  reduce r66: Term -> Factor
22     [0 2 3 4 7 14 15 20 23 27 38 50 52 77 105] [class id { Type id ( ParamListOpt ) { print ( AddExpr
AddOp Term] [* number ) ; return ; } } ...]  shift 87
23     [0 2 3 4 7 14 15 20 23 27 38 50 52 77 105 87] [class id { Type id ( ParamListOpt ) { print (
AddExpr AddOp Term *] [number ) ; return ; } } $]  reduce r49: MulOp -> *
24     [0 2 3 4 7 14 15 20 23 27 38 50 52 77 105 89] [class id { Type id ( ParamListOpt ) { print (
AddExpr AddOp Term MulOp] [number ) ; return ; } } $]  shift 60
25     [0 2 3 4 7 14 15 20 23 27 38 50 52 77 105 89 60] [class id { Type id ( ParamListOpt ) { print (
AddExpr AddOp Term MulOp number] [] ; return ; } } $]  reduce r53: Factor -> number
26     [0 2 3 4 7 14 15 20 23 27 38 50 52 77 105 89 107] [class id { Type id ( ParamListOpt ) { print (
AddExpr AddOp Term MulOp Factor] [] ; return ; } } $]  reduce r65: Term -> Term
27     [0 2 3 4 7 14 15 20 23 27 38 50 52 77 105] [class id { Type id ( ParamListOpt ) { print ( AddExpr
AddOp Term] [] ; return ; } } $]  reduce r63: AddExpr -> AddExpr
28     [0 2 3 4 7 14 15 20 23 27 38 50 52] [class id { Type id ( ParamListOpt ) { print ( AddExpr] [] ;
return ; } } $]  reduce r62: Expr -> AddExpr
29     [0 2 3 4 7 14 15 20 23 27 38 50 73] [class id { Type id ( ParamListOpt ) { print ( Expr] [] ;
return ; } } $]  shift 103
30     [0 2 3 4 7 14 15 20 23 27 38 50 73 103] [class id { Type id ( ParamListOpt ) { print ( Expr )] [] ;
return ; } } $]  shift 114
31     [0 2 3 4 7 14 15 20 23 27 38 50 73 103 114] [class id { Type id ( ParamListOpt ) { print ( Expr )
;} [return ; } } $]  reduce r27: Stmt -> print

```

```

32  [0 2 3 4 7 14 15 20 23 27 33] [class id { Type id ( ParamListOpt ) { Stmt} [return ; } } $] shift
39
33  [0 2 3 4 7 14 15 20 23 27 33 39] [class id { Type id ( ParamListOpt ) { Stmt return} [; } } $]
reduce r35: ExprOpt -> epsilon
34  [0 2 3 4 7 14 15 20 23 27 33 39 54] [class id { Type id ( ParamListOpt ) { Stmt return ExprOpt} [;
} } $] shift 85
35  [0 2 3 4 7 14 15 20 23 27 33 39 54 85] [class id { Type id ( ParamListOpt ) { Stmt return ExprOpt
;} [} } $] reduce r22: Stmt -> return
36  [0 2 3 4 7 14 15 20 23 27 33 33] [class id { Type id ( ParamListOpt ) { Stmt Stmt} [} } $] reduce
r19: StmtList -> epsilon
37  [0 2 3 4 7 14 15 20 23 27 33 33 42] [class id { Type id ( ParamListOpt ) { Stmt Stmt StmtList} [}
} $] reduce r18: StmtList -> Stmt
38  [0 2 3 4 7 14 15 20 23 27 33 42] [class id { Type id ( ParamListOpt ) { Stmt StmtList} [} } $]
reduce r18: StmtList -> Stmt
39  [0 2 3 4 7 14 15 20 23 27 34] [class id { Type id ( ParamListOpt ) { StmtList} [} } $] shift 43
40  [0 2 3 4 7 14 15 20 23 27 34 43] [class id { Type id ( ParamListOpt ) { StmtList } } $] reduce
r17: Block -> {
41  [0 2 3 4 7 14 15 20 23 26] [class id { Type id ( ParamListOpt ) Block} [} $] reduce r7:
MemberAfter -> (
42  [0 2 3 4 7 14 17] [class id { Type id MemberAfter} [} $] reduce r4: Member -> Type
43  [0 2 3 4 5] [class id { Member} [} $] reduce r3: MemberList -> epsilon
44  [0 2 3 4 5 12] [class id { Member MemberList} [} $] reduce r2: MemberList -> Member
45  [0 2 3 4 6] [class id { MemberList} [} $] shift 13
46  [0 2 3 4 6 13] [class id { MemberList } } $] reduce r1: Program -> class
47  [0 1] [Program] [$] accept
LR parser: ACCEPT

```

6.5 Symbol Table Output

When semantic analysis is enabled on `hello.mj`:

Symbol table dump

Name	Kind	Type	Scope	Line
main	function	void	0	2
Hello	class	class	0	1

7 Limitations

- The compiler is a front-end only; it does not generate intermediate representation or target machine code.
- Semantic checking ensures proper scope rules and declarations but does not implement comprehensive type checking for complex expressions.
- Error recovery uses basic panic-mode techniques (skipping tokens until a synchronization point). More advanced phrase-level recovery is limited.

8 References

1. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Pearson.
2. Course Materials and Lab Manuals provided by Instructor Aliya Farooq.